



VIVO v25

Masternode & EVO Masternode Operator Guide

Deterministic ProTx setup • VPS configuration • Security • Verification

Viva la VIVO

Release-focused draft for the new Dash-derived VIVO codebase

1. Purpose and authority

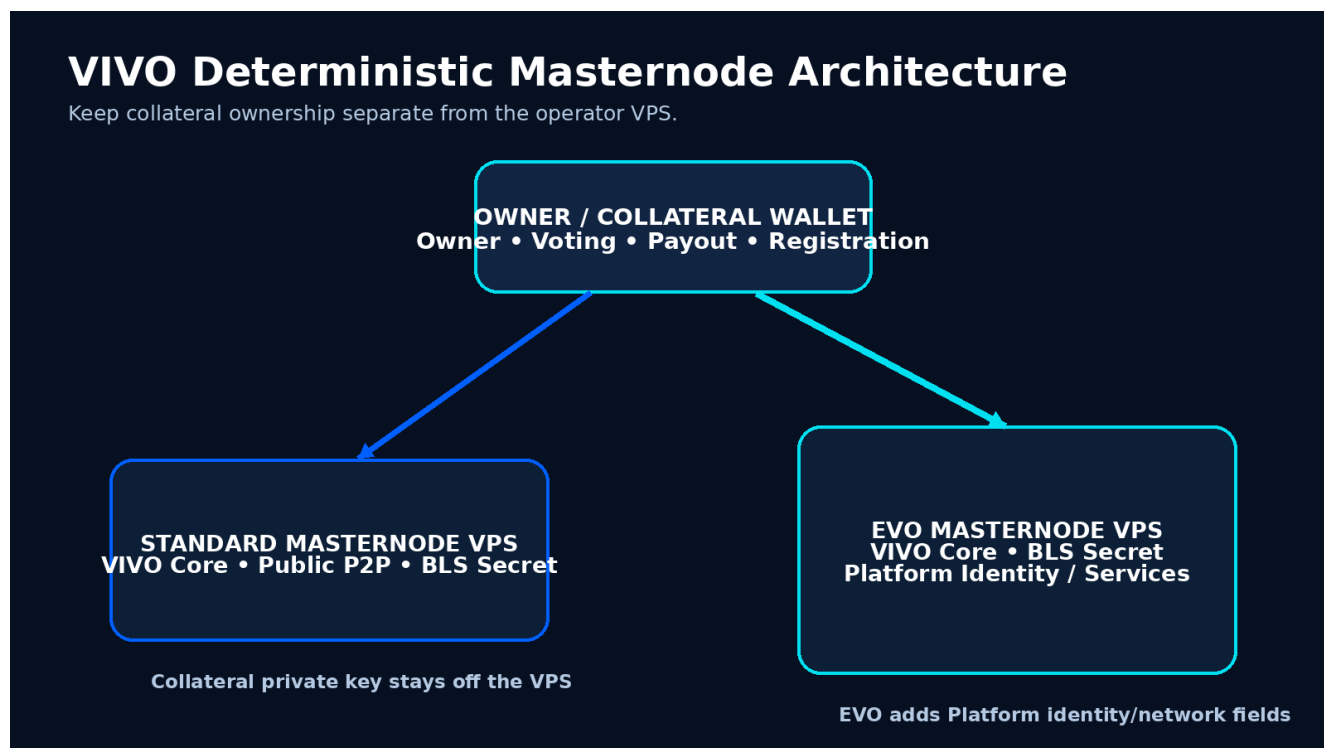
This guide explains how to deploy a standard VIVO deterministic Masternode and an EVO Masternode using the new VIVO v25-style codebase. It intentionally avoids the old legacy masternode.conf workflow once deterministic masternodes are active.

Authority order

For any value that can affect funds or consensus, trust the compiled VIVO consensus code first, then the output of vivo-cli help, then this document. Do not guess collateral amounts, ports, activation heights, or ProTx argument order.

What changes with deterministic masternodes?

- Registration is recorded on-chain with a Provider Registration Transaction (ProRegTx).
- The owner/collateral wallet can stay off the VPS.
- The VPS primarily needs the BLS operator secret and a reachable Core P2P service.
- EVO nodes add Platform/EVO identity and service fields beyond a standard Masternode.



2. Release and activation checks

Before funding any collateral, confirm the exact binary and activation state on the live chain.

```
vivo-cli --version
vivo-cli getnetworkinfo
vivo-cli getblockchaininfo
vivo-cli help protx
```

Current project targets to verify

V2 activation has been designed around block 2,176,696. If DIP0003 remains configured 500 blocks later, the corresponding height is 2,177,196. Treat those as project targets until confirmed in the final compiled release.

```
vivo-cli help protx register
vivo-cli help protx register_prepare
vivo-cli help protx register_submit
vivo-cli help protx register_evo
vivo-cli help protx register_prepare_evo
vivo-cli help protx update_service
vivo-cli help protx update_service_evo
```

3. Standard Masternode vs EVO Masternode

Capability	Standard MN	EVO MN
Collateral	Exact standard MN collateral from VIVO consensus	Exact EVO collateral from VIVO consensus
On-chain identity	ProRegTx	EVO/HPMN-style ProRegTx
Operator key	BLS operator key	Dedicated BLS operator key
Core P2P	Required	Required
Platform Node ID	No	Yes, if enabled by VIVO implementation
Platform P2P / HTTP	No	Yes, if enabled by VIVO implementation
Owner wallet on VPS	Not recommended	Not recommended

Planned EVO collateral

The VIVO redesign has used 20,000 VIVO as the intended EVO collateral. Verify that the final v25 consensus code actually enforces 20,000 before creating the UTXO.

4. Shared prerequisites

- A fully synchronized VIVO v25 node and a safe owner wallet.
- A Linux VPS with a static public IPv4 address.
- The correct VIVO mainnet Core P2P port open to the Internet.
- RPC bound only to localhost; never expose VIVO RPC publicly.
- Exact collateral UTXO plus a small additional balance for transaction fees.
- Backups of owner, voting and payout keys/addresses before registration.

```
# Check synchronization
vivo-cli getblockchaininfo

# Locate mainnet P2P defaults in source
grep -Rni 'nDefaultPort' src/chainparams.cpp
```

5. Standard VIVO Masternode - registration flow

Standard Masternode Registration Flow



Authority order: compiled VIVO consensus code → vivo-cli help → this guide.

Never guess collateral, ports, activation heights, or ProTx argument order.

5.1 Create the collateral UTXO

```
vivo-cli getnewaddress "VIVO-MN-COLLATERAL"
```

Send the exact standard Masternode collateral required by the compiled VIVO code to that address. Then locate the correct output index (vout).

```
vivo-cli getrawtransaction COLLATERAL_TXID true
```

Do not spend this UTXO

The collateral output must remain unspent while the Masternode exists. Use a dedicated collateral address rather than an everyday payment address.

5.2 Create owner, voting, payout and fee addresses

```
vivo-cli getnewaddress "VIVO-MN-OWNER"
vivo-cli getnewaddress "VIVO-MN-VOTING"
vivo-cli getnewaddress "VIVO-MN-PAYOUT"
vivo-cli getnewaddress "VIVO-MN-FEES"
```

Fund the fee address with a small amount of VIVO sufficient to pay the registration transaction fee.

5.3 Generate the BLS operator key

```
vivo-cli bls generate
```

- BLS secret/private key: goes on the Masternode VPS.
- BLS public key: goes into the on-chain ProRegTx registration.
- Never reuse one BLS operator key across independent Masternodes.

6. Standard Masternode VPS configuration

6.1 Install and identify the release

```
install -m 0755 vivod /usr/local/bin/vivod
install -m 0755 vivo-cli /usr/local/bin/vivo-cli

vivod --version
vivo-cli --version
```

6.2 Example vivo.conf

```
server=1
daemon=1
listen=1

rpcbind=127.0.0.1
rpcallowip=127.0.0.1

masternode=1
externalip=YOUR_PUBLIC_IP:VIVO_P2P_PORT
masternodeblsprivkey=YOUR_BLS_SECRET
```

Check externalip syntax

Confirm whether the final VIVO binary expects IP only or IP:PORT with: `vivod -help | grep -i externalip`

6.3 Firewall and synchronization

```
ufw allow VIVO_P2P_PORT/tcp

# RPC should remain local-only
vivo-cli getblockchaininfo
```

Do not continue until `initialblockdownload` is false and the node is synchronized to the active chain tip.

7. Register the standard Masternode

First inspect the exact parameter order used by the final binary:

```
vivo-cli help protx register_prepare
```

A modern address-array form may look conceptually like this:

```
vivo-cli protx register_prepare \
  "$COLLATERAL_TXID" \
  "$COLLATERAL_VOUT" \
  '["YOUR_PUBLIC_IP:VIVO_P2P_PORT"]' \
  "$OWNER_ADDRESS" \
  "$BLS_PUBLIC" \
  "$VOTING_ADDRESS" \
  0 \
  "$PAYOUT_ADDRESS" \
  "$FEE_ADDRESS"
```

Operator reward is not the block reward split

The `operatorReward` field controls whether the owner shares part of the Masternode payment with a separate operator. It does not define the VIVO miner/Masternode consensus split.

7.1 Sign and submit

```
vivo-cli signmessage "$COLLATERAL_ADDRESS" "$SIGN_MESSAGE"
```

```
vivo-cli protx register_submit "$TX" "$SIGNATURE"
```

Record the returned ProTx hash. It is the deterministic Masternode identity and is different from the collateral transaction ID.

8. Verify the standard Masternode

```
vivo-cli protx info "$PROTX_HASH"  
vivo-cli protx list valid true  
vivo-cli masternode status
```

- Correct collateral and output index.
- Correct service IP / Core P2P endpoint.
- Correct owner, voting and payout addresses.
- BLS operator public key matches the secret installed on the VPS.
- Node is valid and not PoSe banned.

9. VIVO EVO Masternode - registration flow

EVO Masternode Registration Flow



Authority order: compiled VIVO consensus code → vivo-cli help → this guide.

Never guess collateral, ports, activation heights, or ProTx argument order.

9.1 Confirm EVO support in the exact binary

```
vivo-cli help protx register_evo
vivo-cli help protx register_prepare_evo
vivo-cli help protx update_service_evo
```

EVO is implementation-sensitive

The final VIVO build determines whether EVO registration requires Platform Node ID, Platform P2P address/port, Platform HTTPS address/port, or placeholders. Follow vivo-cli help exactly.

9.2 Create the EVO collateral UTXO

```
vivo-cli getnewaddress "VIVO-EVO-COLLATERAL"
```

Send the exact EVO collateral enforced by the VIVO consensus code. If the planned 20,000 VIVO value remains in the final build, create an output of exactly 20,000 VIVO.

```
vivo-cli getrawtransaction EVO_COLLATERAL_TXID true
```

9.3 Create EVO owner, voting and payout addresses

```
vivo-cli getnewaddress "VIVO-EVO-OWNER"
vivo-cli getnewaddress "VIVO-EVO-VOTING"
vivo-cli getnewaddress "VIVO-EVO-PAYOUT"
```

9.4 Generate a dedicated EVO BLS key

```
vivo-cli bls generate
```

Do not reuse the BLS key from a standard Masternode.

10. EVO Platform identity and ports

Inspect the source and RPC help before generating or assigning any Platform values:

```
vivo-cli help protx register_evo
```

```
grep -RniE 'platformNodeID|platformP2P|platformHTTP|register_evo' src | head -200
```

```
grep -RniE 'platform.*port|Platform.*Port' src | head -100
```

Item	Value / Record
Platform Node ID	<generated unique EVO Platform Node ID>
Platform P2P endpoint	YOUR_EVO_PUBLIC_IP:26656
Platform HTTPS endpoint	YOUR_EVO_PUBLIC_IP:443
EVO Core P2P endpoint	YOUR_EVO_PUBLIC_IP:12845

Do not copy Dash ports blindly

Even though VIVO is Dash-derived, VIVO should use the values defined by its own source and release configuration.

11. Configure and register the EVO Masternode

11.1 Core VPS configuration

```
server=1
daemon=1
listen=1
rpcbind=127.0.0.1
rpccallowip=127.0.0.1
masternode=1
externalip=YOUR_EVO_PUBLIC_IP:VIVO_P2P_PORT
masternodeblsprivkey=YOUR_EVO_BLS_SECRET
```

11.2 Registration structure

The exact syntax is release-dependent. Conceptually an EVO registration can contain:

- Collateral transaction hash and output index.
- Core P2P endpoint.
- Owner address, BLS operator public key and voting address.
- Operator reward and payout address.
- Platform Node ID.
- Platform P2P and HTTPS endpoints.
- Fee source address.

```
# Always inspect first
vivo-cli help protx register_evo
vivo-cli help protx register_prepare_evo
```

11.3 Preferred prepare / sign / submit flow

```
vivo-cli protx register_prepare_evo ...EXACT_ARGUMENTS_FROM_HELP...
vivo-cli signmessage "$EVO_COLLATERAL_ADDRESS" "$SIGN_MESSAGE"
vivo-cli protx register_submit "$TX" "$SIGNATURE"
```

Record the EVO ProTx hash and preserve it with the owner records.

12. Verify the EVO Masternode

```
vivo-cli protx info "$EVO_PROTX_HASH"
vivo-cli protx list valid true
vivo-cli masternode status
```

- Correct EVO node type and collateral.
- Correct Core service endpoint.
- Correct BLS operator key.
- Correct Platform Node ID and Platform endpoints when enabled.
- Valid registration and no PoSe ban.

13. Updating an existing deterministic node

Item	Value / Record
------	----------------

Change service IP / Core endpoint	protx update_service
Change EVO/Platform service data	protx update_service_evo (if implemented)
Change operator key / voting / payout	protx update_registrar
Revoke compromised operator	protx revoke

```
vivo-cli help protx update_service  
vivo-cli help protx update_service_evo  
vivo-cli help protx update_registrar  
vivo-cli help protx revoke
```

14. Recommended systemd service

```
[Unit]
Description=VIVO Core Daemon
After=network-online.target
Wants=network-online.target

[Service]
Type=forking
User=vivo
Group=vivo
ExecStart=/usr/local/bin/vivod -daemon -datadir=/home/vivo/.vivocore
ExecStop=/usr/local/bin/vivo-cli -datadir=/home/vivo/.vivocore stop
Restart=on-failure
RestartSec=10
PrivateTmp=true
ProtectSystem=full
NoNewPrivileges=true
TimeoutStartSec=120
TimeoutStopSec=120

[Install]
WantedBy=multi-user.target

systemctl daemon-reload
systemctl enable vivod
systemctl start vivod
systemctl status vivod --no-pager
journalctl -u vivod -n 100 --no-pager
```

15. Security model

Secret / credential	Where it should live	Why
Collateral private key	Owner wallet / secure backup	Controls the collateral funds
Owner private key	Owner wallet / secure backup	Controls important deterministic MN properties
Voting private key	Owner/governance wallet	Controls governance voting if enabled
BLS operator secret	Only the intended VPS	Allows the node to operate; does not need collateral custody

RPC rule

Bind RPC to 127.0.0.1 and do not expose it to the public Internet. Only the intended P2P/service ports should be reachable externally.

16. Troubleshooting

Node does not appear in deterministic list

- Confirm DIP0003 / deterministic registration is active.
- Confirm the ProRegTx has confirmations.
- Confirm collateral amount and vout are correct.

- Confirm the collateral UTXO remains unspent.

PoSe banned or unreachable

- Verify public P2P reachability and firewall rules.
- Verify externalip matches the ProTx service endpoint.
- Verify the BLS secret matches the registered operator public key.
- Verify the daemon is synchronized and system time is correct.

EVO registration rejected

- Re-check exact EVO collateral.
- Re-check EVO node type and activation state.
- Re-check Platform Node ID and endpoint formats.
- Re-check argument count and order from vivo-cli help protx register_evo.

17. Standard Masternode worksheet

Item	Value / Record
VIVO version	_____
DIP0003 active height	_____
Standard MN collateral	_____ VIVO
Public IP / Core P2P port	_____
Collateral address	_____
Collateral TXID	_____
Collateral vout	_____
Owner address	_____
Voting address	_____
Payout address	_____
Fee address	_____
BLS public key	_____
ProTx hash	_____

18. EVO Masternode worksheet

Item	Value / Record
VIVO version	_____
EVO collateral	_____ VIVO
Public IP / Core P2P port	_____
EVO collateral address	_____
EVO collateral TXID / vout	_____
Owner address	_____
Voting address	_____
Payout address	_____
BLS public key	_____
Platform Node ID	_____
Platform P2P endpoint	_____
Platform HTTP endpoint	_____
EVO ProTx hash	_____

19. Pre-registration safety checklist

- ☐ Correct VIVO v25 binary installed and hash verified
- ☐ Mainnet selected and blockchain synchronized
- ☐ V2 / DIP0003 activation confirmed
- ☐ Exact collateral amount confirmed in final code
- ☐ Collateral UTXO created and sufficiently confirmed
- ☐ Collateral UTXO unspent
- ☐ Static public IP configured
- ☐ Core P2P port externally reachable
- ☐ RPC inaccessible from public Internet
- ☐ Fresh BLS key generated for this node
- ☐ BLS secret installed only on intended VPS
- ☐ BLS public key matches registration
- ☐ Owner, voting and payout addresses backed up
- ☐ Fee source funded
- ☐ Exact register_* syntax confirmed with vivo-cli help

EVO-only checks

- ☐ EVO collateral type/value confirmed
- ☐ register_evo available and activated
- ☐ Platform Node ID format confirmed
- ☐ Platform P2P / HTTP formats and ports confirmed
- ☐ EVO service endpoints reachable as required

20. Post-registration health check

```
echo "=== VERSION ==="
vivo-cli getnetworkinfo

echo "=== CHAIN ==="
vivo-cli getblockchaininfo

echo "=== MASTERNODE ==="
vivo-cli masternode status

echo "=== VALID DETERMINISTIC NODES ==="
vivo-cli protx list valid true
```

Final certification

Before publishing the guide as authoritative, run the help commands in this document against the final VIVO v25.0.0.0 mainnet binary and replace every remaining placeholder with the exact values enforced by that release.



VIVO

Deterministic ownership. Separate operator security. Clear on-chain registration.

Viva la VIVO